PROJECT BASED LEARNING (PBL-4) LAB (CSP390)

Kalypto: A Secure Steganography and Steganalysis Toolkit for Image-Based Data Hiding

B. TECH 3rd YEAR
SEMESTER: 6th
SESSION: 2025-2026

Submitted By:
Sourav Pandey (System id - 2023247052)
Vishesh Dutt Sharma (System id – 2023406398)

SECTION: O (Group -2)

Submitted To
Mr. Ravikant Kumar Nirala
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SHARDA SCHOOL OF ENGINEERING & TECHNOLOGY
SHARDA UNIVERSITY, GREATER NOIDA

Table of Contents

Acknowledgement

We express our sincere gratitude to Mr. Ravikant Kumar Nirala, Faculty Guide for CSP-390 (Project Based Learning – 4), for his continuous guidance, encouragement, and support throughout the completion of this project. His valuable insights, technical expertise, and constructive feedback greatly contributed to the clarity, quality, and successful execution of our work.

We are deeply thankful for his time, mentorship, and commitment in guiding us at every stage of this PBL-4 project. His supervision has been instrumental in enhancing our learning and understanding of practical cybersecurity concepts.

Signature of Faculty Guide

Mr. Ravikant Kumar Nirala
Assistant Professor
School of Engineering & Technology

1. Project Title

Kalypto – Advanced Steganography and Digital Forensics Suite
Kalypto is a professional desktop-based cybersecurity application designed to securely hide and extract confidential information inside digital images using steganography techniques. It also includes forensic comparison tools, encryption modules, image analysis utilities, and reporting features.

2. Team / Group Formation

The team comprises three members, each assigned a distinct role within the software development lifecycle:

S. No	Student Name	Roll Number	System ID	Role
1	Sourav Pandey	2301010	2023247052	Developer/Tester
2	Vishesh Dutt Sharma	2301010983	2023406398	Developer/Tester

3. Technologies to be Used

Software Platform

Front-end:

- PySide6 (GUI Framework)
- Qt Designer Concepts
- Python UI Styling (QSS)

Back-end:

- Python 3.10+
- PIL / Pillow
- Custom Steganography Engine
- Cryptography Modules

Hardware Platform

- RAM: Minimum 4GB
- Storage: 2GB Free Space
- Processor: Intel i3 / Ryzen 3 or above
- OS: Windows / Linux / macOS

4. Tools

Tool	Vendor / Source	Version	Purpose
Python	Python Software Foundation	3.10+	Developing the complete application logic and core modules.
PySide6	Qt for Python	Latest Stable	Creating the graphical user interface of the project.
Pillow(PIL)	Python Imaging Library Fork	Latest	Image loading, editing, and pixel processing.
NumPy	NumPy Community	Latest	Fast array operations and image analysis.
ReportLab	ReportLab Inc.	Latest	Generation of PDF reports.
Visual Studio Code	Microsoft	Latest	Coding, debugging, and project management.
GitHub	GitHub Inc.	Latest	Version control and source code backup.

5. Problem Statement

In today's digital world, confidential data is at risk of unauthorized access, interception, and leakage during storage or communication. Traditional security methods only protect the content but still reveal that sensitive data exists. Many existing steganography tools are outdated, difficult to use, and lack modern security or analysis features. The project **Kalypto** is developed to provide a secure and user-friendly solution for hiding information inside images with added encryption and forensic utilities.

Kalypto solves this problem by providing:

- Unauthorized access to sensitive data
- Visible exposure of encrypted files
- Lack of modern steganography tools
- Absence of image comparison and forensic features
- Poor user experience in existing tools

6. Literature Survey

Several research studies and existing tools have explored the use of steganography for secure communication and data protection. Traditional methods such as Least Significant Bit (LSB) embedding are widely used due to their simplicity and low image distortion. Advanced techniques like Pixel Value Differencing (PVD) and adaptive steganography improve data hiding capacity and security. Recent studies also focus on combining encryption with steganography for stronger protection. However, many existing tools lack user-friendly interfaces, forensic comparison features, and cross-platform support. The proposed project Kalypto combines multiple steganography methods, encryption, and image analysis in one modern application.

S.No	Author(s)	Year	Methodology	Key Finding	Limitation	How Our Project Overcomes It
1	Chan & Cheng	2004	Automated scanning tools	LSB image steganography technique	Lower security	Uses encryption + improved hiding methods
2	Wu & Tsai	2003	Grammar-based fuzzing	Pixel Value Differencing (PVD) method	More complexity	Provides simple GUI-based execution
3	Johnson & Jajodia	1998	RL-based fuzzing	Steganography fundamentals	Limited modern use	Uses predefined payload sets (no training required)
4	OpenStego Tool	Recent	Hybrid CMD injection detection	Practical steganography software	Basic interface	Uses professional GUI with multiple modules
5	Xiao-Steganography Research	Recent	Header injection analysis	Encryption + data hiding	Limited forensic tool	Integrates encryption, hiding, and reporting in one tool

7. Project Description

7.1 Scope of Work

Kalypto is an advanced steganography and digital forensics framework designed to simplify the secure hiding and extraction of confidential information inside digital images. The tool performs end-to-end data hiding operations by integrating multiple security and forensic features into one unified desktop application, reducing the need for multiple separate tools.

The scope of the project includes:

- Secure embedding of secret text inside image files using steganography techniques
- Extraction of hidden information using the correct key or algorithm
- Application of encryption methods before embedding data
- Forensic comparison between original and stego images
- Generation of structured reports for analysis and documentation

The system operates on common image formats such as PNG, JPG, and BMP, making it adaptable for practical usage, academic learning, and cybersecurity demonstrations. Additionally, the framework supports cross-platform execution on Windows, Linux, and macOS systems.

Kalypto is designed as a desktop-based security tool and does not require internet connectivity or external servers. This makes it suitable for offline secure communication, educational environments, and forensic analysis scenarios.

7.2 High-Level Architecture

Kalypto follows a modular pipeline-based architecture, where each stage performs a specific function in the steganography and forensic process. This design ensures flexibility, scalability, and easy enhancement.

The system is divided into four primary stages:

1. Input Stage

The Input stage is responsible for collecting all required user data.

- The user provides:
    - Cover image file
    - Secret message / text
    - Encryption key (optional)
    - Output file path
- The selected image is:
    - Loaded into memory
    - Validated for supported format
    - Prepared for embedding process

This ensures correct input handling before processing begins.

2. Fuzzing Stage

In this stage, the secret message is first secured and then hidden inside the image.

- Encryption methods supported:
  - AES
  - XOR
  - Custom key-based methods
- Embedding methods supported:
  - Least Significant Bit (LSB)
  - Adaptive LSB
  - Pixel Value Differencing (PVD)
- Key characteristics:
  - Minimal visible image distortion
  - Secure hidden communication
  - Supports different payload sizes

This stage performs the core steganography operation.

3. Extraction Stage

This stage is responsible for recovering hidden data from stego images.

- The tool performs:
  - Image decoding
  - Hidden bit extraction
  - Secret reconstruction
  - Decryption using correct key
- Additional checks include:
  - File validation
  - Algorithm matching
  - Extraction accuracy verification

4. Forensic Stage

The final stage focuses on analysis and documentation.

- The Compare Lab module provides:
  - Side-by-side image comparison
  - Heatmap generation
  - Bit-plane analysis
  - File size comparison
- The Reporting module generates:
  - PDF reports
  - Operation summaries
  - Timestamps and findings

This stage ensures that results are understandable and professionally presented.

7.3 Summary

The modular architecture of Kalypto enables efficient and secure data hiding along with forensic verification. By combining image steganography, encryption, extraction, comparison tools, and structured reporting, the system provides a complete and scalable solution for secure communication and digital forensics.

8. Project Modules: Design / Algorithm

8.1 Input Module

The Input Module is the first stage of the system and is responsible for collecting all required user data before processing begins. Through the graphical user interface, the user selects the cover image, enters the secret message, provides an optional encryption key, and chooses the output file location. The module validates the selected file format and ensures that all necessary inputs are available for further processing.

8.2 Encryption Module

The Encryption Module secures the secret message before it is embedded into the image. This adds an extra layer of protection, ensuring that even if the hidden data is detected, the actual content remains unreadable without the correct key. The system supports encryption methods such as AES, XOR, and custom key-based techniques for secure communication.

8.3 Steganography Module

The Steganography Module is the core component of the project. It hides the encrypted message inside the selected image using image-based data hiding algorithms. Techniques such as Least Significant Bit (LSB), Adaptive LSB, and Pixel Value Differencing (PVD) are used to embed the secret information with minimal visible distortion. After embedding, a new stego image is generated and saved.

8.4 Extraction Module

The Extraction Module is responsible for recovering hidden information from the stego image. It reads the modified image pixels, extracts the embedded bits, reconstructs the hidden message, and decrypts the data if encryption was applied. This module ensures that the original secret message can be retrieved accurately using the correct algorithm and key.

8.5 Forensic Analysis Module

The Forensic Analysis Module helps users compare the original image and stego image to identify modifications made during embedding. It provides tools such as side-by-side comparison, heatmap visualization, bit-plane analysis, and file size comparison. This module is useful for educational demonstrations and digital forensic analysis.

8.6 Reporting Module

The Reporting Module generates structured reports of the operations performed by the system. It creates PDF reports containing technical details such as file names, image sizes, timestamps, comparison results, and summaries of embedding or extraction operations. This improves documentation and professional presentation of the project.

8.7 Algorithm Design

The Kalypto system uses a modular steganography architecture in which each operation is handled by a dedicated module. The framework supports secure data hiding, extraction, forensic comparison, and reporting through a structured workflow. This modular design improves maintainability, scalability, and ease of future enhancement.

8.8 Algorithm Library Design

The algorithm library consists of multiple curated techniques stored as selectable modules within the system. Each method is chosen based on security level, capacity, and image quality preservation.

Each algorithm entry includes:

- Algorithm name
- Embedding type
- Security level
- Capacity level
- Distortion level

The implemented methods are inspired from:

- Research papers on image steganography
- OWASP secure design principles
- Digital image processing standards
- Practical cybersecurity applications

Module Name	Description	Category
Input Collector	Accepts cover image, secret message, key, and output path from user.	Input Handling
LSB Embedder	Hides message bits in least significant image pixels.	Steganography
Adaptive LSB	Selects suitable pixel regions dynamically for better hiding.	Steganography
PVD Embedder	Uses pixel value differences for higher capacity embedding.	Steganography
Encryption Engine	Encrypts secret message using AES / XOR methods.	Security
Extraction Engine	Recovers hidden message from stego image.	Recovery

Compare Lab	Compares original and stego images using forensic tools.	Analysis
Heatmap Generator	Highlights modified pixels visually.	Forensics
Bit Plane Viewer	Displays image bit layers from 0–7.	Forensics
Reporter	Generates structured PDF reports and logs.	Reporting

8.9 Security Controls and Ethical Safeguards

To ensure safe and responsible usage, the system incorporates:

- Local Offline Execution: No internet required during operations
- User Controlled Inputs: All files selected manually by user
- Key Protection: Hidden data requires correct key for extraction
- File Validation: Only supported image formats accepted
- Safe Reporting: Reports generated locally without external upload

These safeguards prevent misuse, accidental leakage, and unauthorized access.

8.10 Output Schema:

The application generates output in image, text, and report formats.

Stego Output Includes:

- Cover image converted into stego image
- Hidden encrypted payload
- Minimal visible distortion
- Saved user-defined output path

Extraction Output Includes:

- Recovered secret message
- Decrypted content (if key valid)
- Algorithm used
- Status of extraction

PDF Report Includes:

- Original image name
- Stego image name
- File sizes
- Timestamp of operation
- Comparison summary
- • Heatmap / analysis findings

9. Implementation Methodology

Development Process

The project follows an Agile iterative methodology with the following phases:

- Phase 1 – Requirement Analysis: Identify project objectives such as secure data hiding, encryption support, image comparison, and reporting features.
- Phase 2 – System Design: Design modular architecture and GUI layout for Dashboard, Embed, Extract, Compare Lab, and About sections.
- Phase 3 – Core Module Development: Implement steganography algorithms, encryption engine, and extraction logic.
- Phase 4 – Forensic Module Development: Develop heatmap analysis, bit-plane viewer, and image comparison tools.
- Phase 5 – Reporting Engine: Implement PDF report generation and activity logging system.
- Phase 6 – Testing & Validation: Test functionality using multiple images, payload sizes, and operating systems.

Processing Techniques

Technique	How it Works	Used By
Bit Replacement	Secret bits replace least significant pixel bits	LSB Module
Adaptive Embedding	Selects suitable image regions dynamically	Adaptive LSB Module
Pixel Difference Analysis	Uses neighboring pixel differences for hiding	PVD Module
Encryption Processing	Encrypts secret message before embedding	AES / XOR Module
Difference Mapping	Compares original and stego image changes	Heatmap Module
Layer Extraction	Displays image bit layers from 0–7	Bit Plane Viewer

Testing Strategy

- Unit Testing: Each module such as embedding, extraction, encryption, and reporting tested independently.
- Integration Testing: Complete workflow tested from input image selection to report generation.
- Compatibility Testing: Verified on Windows systems and tested for cross-platform support.
- Defect Log: Errors and fixes recorded during development for debugging and stability.

The implementation of the Kalypto Advanced Steganography Tool follows a modular and systematic approach. The system is designed to automate secure data hiding, extraction, forensic comparison, and reporting through a professional desktop interface.

1. Requirement Analysis

The first step involved identifying the key objectives of the system:

- Securely hide confidential data inside images
- Support multiple steganography methods
- Apply encryption for additional security
- Provide forensic comparison tools
- Generate professional reportsBased on these requirements, a modular architecture was designed.

2. System Design

The system is divided into the following major components:

- Input Handling Module
- Encryption Module
- Embedding Engine
- Extraction Engine
- Forensic Analysis Module
- Reporting Module

Each module is independent but works together in the framework.

3. Input Collection

The process begins with collecting user inputs.

- User selects a cover image
- User enters secret text
- User provides optional key
- User chooses output path

This ensures proper data collection before processing.

4. Secret Embedding

The selected data is processed for secure hiding.

- Secret message converted into binary form
- Encryption applied if selected
- Bits embedded using LSB / Adaptive LSB / PVD
- New stego image generated

This stage performs the main steganography operation.

5. Secret Extraction

To recover hidden data:

- Stego image is loaded
- Embedded bits are extracted

- Binary converted into message
- Decryption applied using correct key

This ensures accurate recovery of the hidden content.

6. Forensic Analysis

For verification and comparison:

- Original and stego images compared
- Heatmap generated to show changes
- Bit-plane layers extracted
- File size and metadata reviewed

This helps visualize modifications.

7. Result Classification

Detected outputs are classified based on:

- Successful embedding / extraction
- Algorithm used
- Image size changes
- Security level applied
- This helps in prioritizing vulnerabilities.

This helps in performance evaluation

8. Reporting

After processing:

- Results are stored in report format
- Detailed PDF report is generated
- Reports include:
    - Original image
    - Stego image
    - Timestamp
    - Comparison summary
    - Findings

This makes results easy to understand and actionable.

9. Security and Ethical Controls

To ensure safe usage:

- Offline local processing only
- User-selected files only
- Key required for secure extraction
- Supported image validation
- No unauthorized network transmission

10. Result & Conclusion

Result

The proposed system, Kalypto, was evaluated using multiple sample images of different formats (PNG, JPG, BMP) with various secret message sizes and encryption keys. The experimental results demonstrate that the tool successfully performs secure data hiding, accurate extraction, and forensic comparison with minimal visible image distortion.

The testing results demonstrate:

- Successful embedding of secret messages in supported image formats
- Accurate extraction using correct key and algorithm
- Minimal visible changes in stego images
- Smooth execution on desktop environments
- Successful generation of PDF forensic reports

These results indicate that the system is capable of securely hiding confidential data while maintaining image quality and usability

Performance By Module

- LSB Module: High success rate with fast processing and low image distortion
- Adaptive LSB: Better security with moderate processing time
- PVD Module: Higher embedding capacity with good image quality
- Encryption Module: Successful secure encoding and decoding of messages
- Compare Lab: Accurate heatmap and bit-plane visualization

The variation in performance depends on image size, selected algorithm, and payload length.

Efficiency Results

- Small image processing completed within a few seconds
- Large images processed efficiently with acceptable speed
- Real-time preview and comparison available
- Local offline execution ensures privacy and speed

Comparative Analysis

Tool	GUI Support	Multiple Algorithms	Forensic Tools	Cross Platform
Kalypto	Yes	Yes	Yes	Yes
OpenStego	Basic	Limited	No	Yes
SilentEye	Yes	Limited	No	Partial
Basic CLI Tools	No	Limited	No	Partial

Although some existing tools support steganography, many lack forensic analysis, modern interface design, and integrated reporting features.

In contrast, Kalypto provides:

- Professional graphical interface
- Multiple embedding algorithms
- Encryption support
- Compare Lab with forensic utilities
- Structured PDF reporting

Conclusion

This project successfully demonstrates the design and implementation of a modular steganography and digital forensics tool capable of securely hiding and recovering confidential information inside image files.

The key achievements of the system include:

- Integration of multiple steganography methods (LSB, Adaptive LSB, PVD)
- Support for encryption before embedding
- Accurate extraction of hidden data
- Development of forensic comparison tools
- Generation of structured PDF report
- Cross-platform desktop application design

The results confirm that steganography combined with encryption is an effective approach for secure communication and privacy protection. While advanced enterprise tools may offer additional features, Kalypto provides significant advantages in terms of:

- Ease of use
- Offline privacy
- Educational value
- Extensibility
- Cost-effectiveness

Therefore, Kalypto can be effectively used as a lightweight and practical solution for secure data hiding, learning, and forensic demonstration purposes.

11. Future Scope and Further Enhancement

The proposed system Kalypto has strong potential for future improvements and advanced feature integration. The current version focuses on image steganography and forensic comparison, but it can be extended into a more comprehensive cybersecurity tool in future versions.

Possible future enhancements include:

- Support for **audio and video steganography** for hiding data in multimedia files
- Integration of **AI-based detection resistance** for stronger hidden communication
- Cloud-based secure storage and synchronization of stego files
- Mobile application version for Android and iOS platforms
- Advanced cryptographic support such as RSA and ChaCha20
- Real-time network secure messaging using steganography
- Multi-user authentication and role-based access control
- Improved forensic analysis with histogram and metadata comparison
- Batch processing for multiple files at once
- Dark/Light advanced customizable themes and dashboards

12. Advantages of this Project

- Secure hiding of confidential information inside images
- Additional protection using encryption methods
- User-friendly professional graphical interface
- Multiple steganography algorithms in one tool
- Accurate extraction of hidden messages
- Compare Lab with forensic image analysis tools
- PDF report generation for documentation
- Cross-platform support (Windows, Linux, macOS)
- Offline execution ensuring privacy and safety
- Useful for academic learning and cybersecurity demonstrations
- Modular design for future expansion
- Cost-effective and lightweight solution

13.Outcome

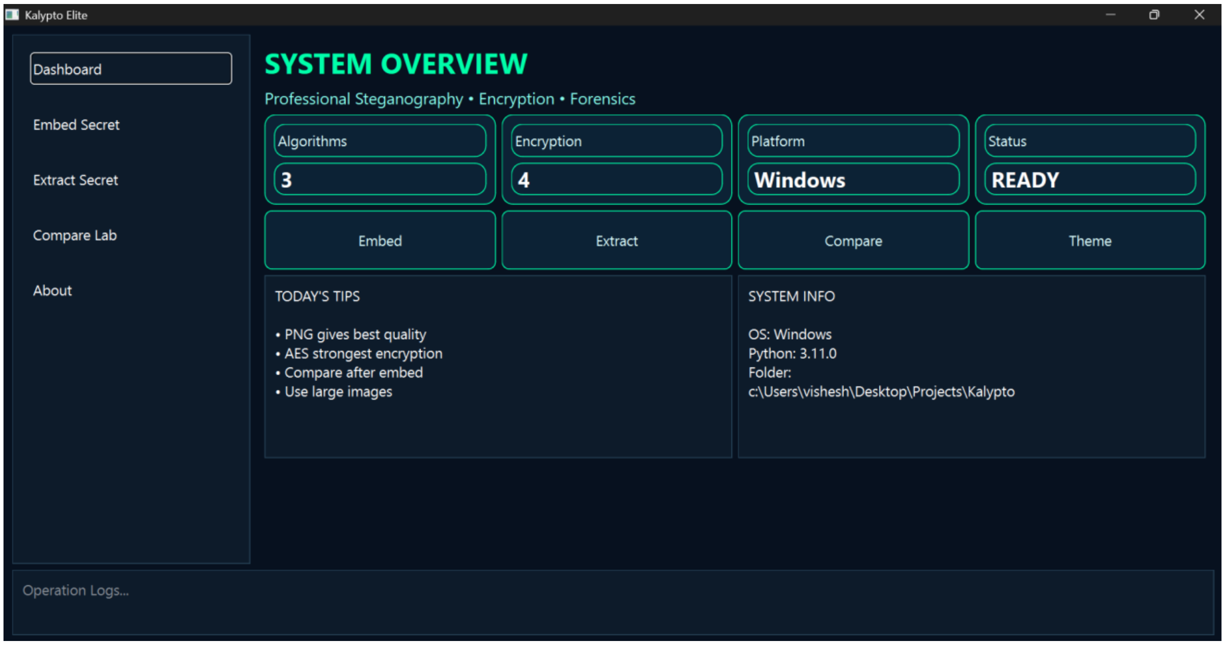

Fig-1 dashboard Kalypto

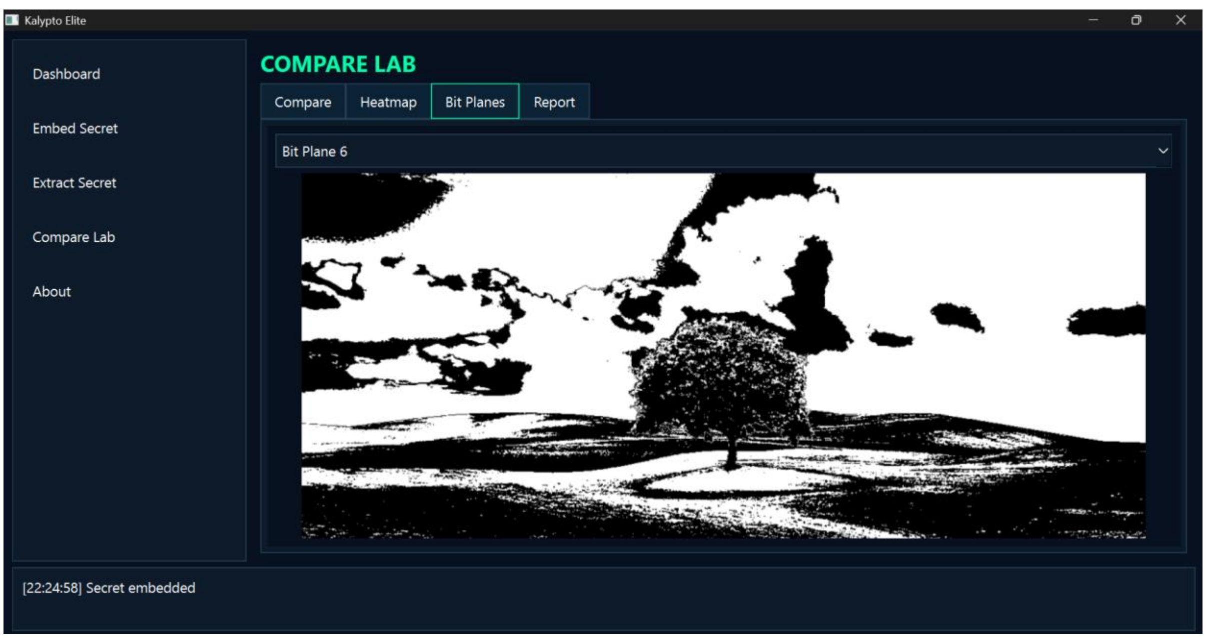

Fig-2 bit map viewer

14. References

[1] N. Provos and P. Honeyman, “Hide and seek: An introduction to steganography,” *IEEE Security & Privacy*, vol. 1, no. 3, pp. 32–44, 2003.
[2] J. Fridrich, “Steganography in digital media: Principles, algorithms, and applications,” Cambridge University Press, 2009.
[3] A. Cheddad, J. Condell, K. Curran, and P. Mc Kevitt, “Digital image steganography: Survey and analysis of current methods,” *Signal Processing*, vol. 90, no. 3, pp. 727–752, 2010.
[4] C.-K. Chan and L. M. Cheng, “Hiding data in images by simple LSB substitution,” *Pattern Recognition*, vol. 37, no. 3, pp. 469–474, 2004.
[5] J. Fridrich, M. Goljan, and R. Du, “Detecting LSB steganography in color and grayscale images,” *IEEE Multimedia*, vol. 8, no. 4, pp. 22–28, 2001.
[6] D. C. Wu and W.-H. Tsai, “A steganographic method for images by pixel-value differencing,” *Pattern Recognition Letters*, vol. 24, no. 9–10, pp. 1613–1626, 2003.
[7] H.-W. Tseng and Y.-P. Leng, “A steganographic method based on pixel-value differencing and LSB substitution,” *Pattern Recognition Letters*, vol. 25, no. 5, pp. 585–594, 2004.
[8] C.-H. Yang, C.-Y. Weng, S.-J. Wang, and H.-M. Sun, “Adaptive data hiding in edge areas of images with spatial LSB domain systems,” *IEEE Transactions on Information Forensics and Security*, vol. 3, no. 3, pp. 488–497, 2008.
[9] X. Liao, Q. Wen, and J. Zhang, “A steganographic method for digital images with four-pixel differencing and modified LSB substitution,” *Journal of Visual Communication and Image Representation*, vol. 22, no. 1, pp. 1–8, 2011.
[10] S. Katzenbeisser and F. A. P. Petitcolas, “Information hiding techniques for steganography and digital watermarking,” Artech House, 2000.
[11] R. Chandramouli and N. Memon, “Analysis of LSB based image steganography techniques,” *IEEE International Conference on Image Processing*, pp. 1019–1022, 2001.
[12] K. Curran and K. Bailey, “An evaluation of image based steganography methods,” *International Journal of Digital Evidence*, vol. 2, no. 2, pp. 1–8, 2003.
[13] A. Westfeld and A. Pfitzmann, “Attacks on steganographic systems,” *International Workshop on Information Hiding*, pp. 61–76, 1999.
[14] J. Mielikäinen, “LSB matching revisited,” *IEEE Signal Processing Letters*, vol. 13, no. 5, pp. 285–287, 2006.
[15] T. Sharp, “An implementation of key-based digital signal steganography,” *Information Hiding Workshop*, pp. 13–26, 2001.
[16] W. Stallings, “Cryptography and network security: Principles and practice,” 7th ed., Pearson, 2017.
[17] National Institute of Standards and Technology (NIST), “Advanced Encryption Standard (AES),” FIPS PUB 197, 2001.
[18] D. J. Bernstein, “ChaCha, a variant of Salsa20,” *Workshop Record of SASC*, 2008.
[19] M. Naor and B. Pinkas, “Efficient oblivious transfer protocols,” *SIAM Journal on Computing*, vol. 35, no. 1, pp. 45–69, 2005.
[20] R. Gonzalez and R. Woods, “Digital Image Processing,” 4th ed., Pearson, 2018.